

챕터 9. 질문을 제대로 이해 못한다. HyDE와 Multi-Query

이번 챕터가 끝나면

- **Parent Document Retriever**로 조각이 아닌 원본 문서 전체를 참고하게 합니다
- **Contextual Compression**으로 검색 결과에서 핵심 문장만 압축합니다
- **HyDE(Hypothetical Document Embeddings)** 로 "상상 답변을 먼저 만들어 검색"합니다
- **Multi-Query**로 하나의 질문을 여러 관점으로 풀어 재현율을 올립니다
- **약어, 동의어 확장**으로 "WFH" ↔ "재택근무" 같은 단어 차이를 흡수합니다

준비하기. 실습 시작 전 한 번만 설정

1. 실습 폴더 이동

터미널 폴더 이동

```
cd rag-start/ex09
```

파일 구조는 다음과 같습니다.

ex09 디렉토리

```

ex09/
├── tuning/
│   ├── step1_query_rewrite/           # [실습] HyDE / Multi-Query / 약어 확장
│   │   ├── __main__.py
│   │   ├── hyde.py                   # 실험 1-1
│   │   ├── multi_query.py           # 실험 1-2
│   │   ├── abbreviation.py          # 실험 1-3
│   │   ├── experiments.py
│   │   └── data.py
│   └── step2_advanced_retriever/      # [실습] Parent Doc / Compression
│       ├── __main__.py
│       ├── parent_doc.py             # 실험 2-1
│       ├── compression.py            # 실험 2-2
│       ├── self_query.py             # 실험 2-3
│       ├── experiments.py
│       └── data.py

```

2. 실습 환경 구축

터미널 환경 구성. macOS / Linux

```

cd ex09
python3.12 -m venv .venv
source .venv/bin/activate
cp .env.example .env
ollama pull llama3.1:8b
pip install -r requirements.txt

```

터미널 환경 구성. Windows

```

cd ex09
py -3.12 -m venv .venv
.venv\Scripts\activate
copy .env.example .env
ollama pull llama3.1:8b
pip install -r requirements.txt

```

3. 사용할 라이브러리

패키지	역할
<code>langchain</code>	Retriever 인터페이스·LCEL
<code>langchain-community</code>	ParentDocumentRetriever, MultiQueryRetriever
<code>langchain-ollama</code>	LLM 연결 (쿼리 재작성용)
<code>langchain-chroma</code> · <code>chromadb</code>	벡터 DB

4. 실행 순서

1. `python -m tuning.step1_query_rewrite` . HyDE, Multi-Query, 약어 확장
2. `python -m tuning.step2_advanced_retriever` . Parent Doc, Compression

9.1 "휴가"라고 물었는데 "연차유급휴가"를 못 찾는다


 회사원

"휴가 규정 알려줘"

×
복리후생 안내가 나옴

"연차유급휴가 알려줘"

✓
취업규칙 제15조 정확히 검색

"WFH 정책이 뭐야?"

×
관련 문서를 찾지 못했습니다

검색 엔진은 잘 돌아갑니다. **질문을 이해 못 하는 게 문제입니다.**

그림 9-1. 같은 뜻인데 못 찾는다

청킹을 바꾸고 리랭커를 붙이고 하이브리드까지 적용했습니다. 일주일이지났죠. 동료는 "병가 규정"을 물으면 출장 규정이 나오던 시절은 끝났다고 생각했습니다. 모니터에 찍히는 검색 로그는 깔끔했고, 오픈이는 의자에 등을 기대며 커피잔을 들었습니다.

검색 엔진은 손덜 데가 없다.

슬랙 알림이 올렸습니다. 동료가 테스트 결과를 스프레드시트로 정리해 올린 겁니다. "질문 20개 테스트" 제목 아래 빨간 셀이 네 개. 오픈이가 커피잔을 내려놓고 몸을 일으켰습니다.

동료 : "질문 20개 돌려 봤는데, 네 개가 이상해요."

스프레드시트를 열었습니다. 첫 번째 빨간 셀. "휴가 규정 알려줘."

직접 쳐 봤습니다. 커넥트HR 채팅창에 "휴가 규정 알려줘"를 입력하고 엔터를 눌렀더니 "복리후생 안내"가 돌아옵니다. 정작 찾고 싶던 **취업규칙 제15조 연차유급휴가**는 빠져 있었습니다.

잠깐.

"연차유급휴가"를 정확히 쳐 봤습니다. 바로 뜹니다. 15일 부여, 3일 전 서면 신청, 팀장 승인. 원하는 내용이 전부 들어 있었습니다.

같은 뜻인데 단어가 달랐습니다. 문서에는 "연차유급휴가"라고 적혀 있고 동료는 "휴가"라고 물었습니다. 검색 엔진은 "휴가"와 "연차유급휴가"가 같은 뜻이라는 걸 모릅니다.

두 번째 빨간 셀. "WFH 정책이 뭐야?"

"관련 문서를 찾지 못했습니다."

문서에는 "재택근무"라고 적혀 있습니다. WFH가 Work From Home이라는 걸 검색 엔진은 모릅니다. 세 번째 빨간 셀도, 네 번째도 비슷한 패턴이었습니다. 단어가 다르거나, 약어를 쓰거나, 질문이 모호하거나.

사용자가 말한 단어 "휴가"	≠	문서에 적힌 단어 "연차유급휴가"	×
사용자가 말한 단어 "WFH"	≠	문서에 적힌 단어 "재택근무"	×
사용자가 말한 단어 "쉬는 날"	≠	문서에 적힌 단어 "연차유급휴가"	×
사용자가 말한 단어 "연차유급휴가"	=	문서에 적힌 단어 "연차유급휴가"	✓

정확히 같은 단어를 써야만 찾습니다. **검색 엔진이 아니라 질문이 문제입니다.**

그림 9-2. 사용자의 단어와 문서의 단어가 다릅니다

챗터 8에서 검색 엔진 자체를 다 고쳤다고 생각했는데. 엔진이 아무리 좋아도 질문을 이해 못 하면 소용이 없다.

건너편 자리에서 팀장이 모니터를 보며 한마디를 던졌습니다.

팀장 : "사서 귀가 안 열린 거 아니가요?"

오픈이 : "네?"

팀장 : "서가 정리는 잘 해놨잖아요. 그런데 손님이 '쉬는 날'이라고 말하면, 그게 '연차유급휴가' 서랍이란 걸 알아들어야 할 거 아니겠어요."

키보드 위에 올려놓은 손가락이 멈췄습니다. 챗터 8에서 고친 건 서가 정리였습니다. 책장을 다시 짜고 면접관을 붙이고 두 가지 검색을 섞었습니다. 그런데 사서의 **귀**, 그러니까 질문을 알아듣는 능력은 건드리지 않았습니다.

서랍에서 수첩을 꺼냈습니다. 화이트보드 앞에 서기 전에 무엇이 문제였는지부터 다시 적어야 했습니다.

— 문제 정리 —

1. "휴가" ≠ "연차유급휴가"

- 문서 단어와 사용자 단어가 다르면 검색 실패
- 같은 뜻이라는 걸 기계는 모름

2. "WFH" ≠ "재택근무"

- 영문 약어는 한글 문서에 안 걸림
- 사내 용어 사전이 없으면 그냥 놓침

→ **엔진 문제가 아님. 질문 이해 능력의 문제**

9.2 사서의 언어 능력을 키운다

오픈이가 화이트보드 앞으로 걸어갔습니다. 마커 뚜껑을 딸깍 열고 왼쪽 끝에 "사서의 언어 능력"이라고 적었습니다. 팀장이 의자를 돌려 팔짱을 끼었습니다.

팀장 : "사서가 똑똑해지려면 뭘 배워야 할까요?"

오픈이가 "휴가 → 연차유급휴가"를 화이트보드에 적고 잠시 마커를 들고 서 있었습니다. 그러다 하나씩 적어 내려가기 시작했습니다.

첫째, 단어를 번역합니다. "WFH" → "재택근무(Work From Home)". "OT" → "초과근무". 사서가 약어 사전을 가지고 있으면 모르는 단어가 들어와도 원래 뜻으로 바꿔 놓을 수 있습니다.

둘째, 답을 먼저 상상합니다. "쉬는 날 규정이라면... 1년 근속 시 15일 유급휴가 같은 내용이겠지." 그 상상한 답과 비슷한 실제 문서를 찾습니다. 질문으로 검색하는 것보다 답변끼리 비교하는 게 거리가 가까운 경우가 많습니다.

셋째, 질문을 여러 갈래로 바꿉니다. "휴가 규정" 하나로 검색하면 한 방향만 봅니다. "연차 사용 방법", "유급휴가 일수", "휴가 신청 절차"로 펼쳐 놓고 각각 검색하면 놓치는 문서가 줄어듭니다.

넷째, 찾은 조각의 원본 전체를 꺼냅니다. "연차 신청은 3일 전 서면"이라는 짧은 청크가 걸리면, 그 청크가 속한 원 문서(제15~16조) 전체를 반환합니다. 맥락이 살아납니다.

다섯째, 긴 결과를 압축합니다. 부모 문서를 통째로 가져오면 쓸데없는 문장이 섞입니다. 질문과 관련된 문장만 추려서 LLM에 넘깁니다.

다섯 줄을 적고 나서 한 발 물러나 화이트보드를 바라봤습니다.

팀장 : "전부 다 켜면 느려져요. 어디가 아픈지 보고 약을 골라야죠."

도구 상자입니다. 다섯 가지를 전부 쓰는 게 아니라, 어떤 질문이 실패하는지 보고 필요한 도구만 꺼내면 됩니다.



질문을 알뜰히 귀

1

단어를 번역한다 — 약어·동의어 확장

WFH → 재택근무 OT → 초과근무

2

답을 먼저 상상한다 — HyDE

"쉬는 날 규정" → "1년 근속 시 15일 유급휴가..." → 상상 답변으로 검색

3

질문을 여러 갈래로 — Multi-Query

"휴가 규정" → "연차 사용 방법" + "유급휴가 일수" + "휴가 신청 절차"



결과를 정리하는 손

4 조각의 원본 전체를 꺼낸다 — **Parent Doc**
"연차 신청은 3일 전 서면" 조각 → 제15~16조 원문 전체 반환

5 핵심 문장만 압축한다 — **Compression**
부모 문서에서 질문과 관련된 문장만 남기고 나머지 제거

1~3(파란색) = 검색 전 질문 가공 · 4~5(초록색) = 검색 후 결과 가공

그림 9-3. 사서의 언어 능력 다섯 가지

9.3 이번 챕터의 두 묶음

다섯 가지 기법이 두 묶음으로 나뉩니다.

검색 전 질의 변환 (Step 1). HyDE, Multi-Query, 약어 확장. 검색 전에 질문 자체를 가공합니다.

검색 후 결과 가공 (Step 2). Parent Doc, Compression. 검색된 결과를 넓히거나 줄여서 LLM에 넘기기 좋은 형태로 만듭니다.

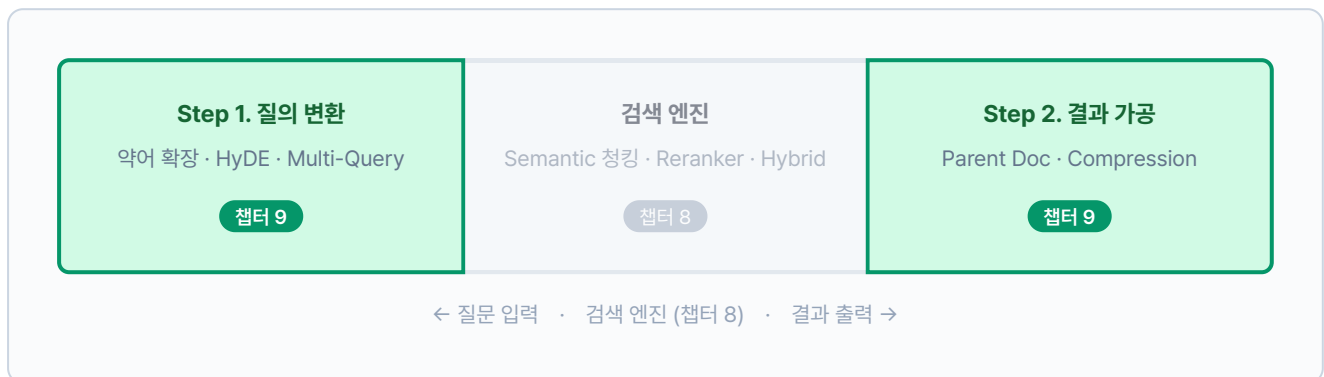


그림 9-4. RAG 파이프라인에서 챕터 8과 챕터 9의 위치

9.4 사서의 귀를 열어 주자 — 질의 변환 (step1)

서가는 정리했습니다. 이번에는 **질문 쪽**을 손봅니다. 검색 엔진에 질문을 넘기기 **전에** 질문 자체를 가공하는 도구 세 가지를 하나씩 실험합니다. 실험 결과를 보고 우리 시스템에 필요한 것만 골라 적용할 겁니다.

실습 흐름

1. 가상 답변으로 검색한 결과와 직접 검색 결과를 비교합니다 (--step 1-1)
2. 하나의 질문을 여러 표현으로 재작성합니다 (--step 1-2)
3. 쿼리의 약어와 동의어를 확장합니다 (--step 1-3)

9.4.1 답을 먼저 상상한다 — HyDE

답을 먼저 상상해 보면 어떨까.

사서가 "쉬는 날 규정"이라는 질문을 받으면, 서가로 달려가기 전에 먼저 "1년 근속 시 15일 유급휴가..."라는 답을 상상합니다. 그 상상한 답과 비슷한 문서를 찾으면, 질문과 문서 사이의 어휘 차이를 건너뛸 수 있습니다.

질문 대신 **"이 질문에 어울릴 법한 답변"**을 LLM에 먼저 생성시키고, 그 가상 답변을 임베딩해 검색합니다. 답변과 답변 간 거리가 질문-답변 거리보다 가까운 경우가 많기 때문입니다.



핵심: 답변 ↔ 문서 거리 < 질문 ↔ 문서 거리. 가상 답변이 어휘 차이를 메워 줍니다.

그림 9-5. 직접 검색과 HyDE 검색의 차이

이 흐름을 코드로 구현합니다. 질문 벡터와 가상 답변 벡터를 각각 만들고, 문서 벡터와의 유사도를 나란히 비교하는 함수입니다.

`tuning/step1_query_rewrite/hyde.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_query_rewrite/hyde.py. HyDE vs 직접 검색 비교

```
def compare_hyde_vs_direct(query, documents, embeddings=None):
    # TODO: 가상 답변으로 검색한 결과와 직접 검색 결과를 비교합니다.

    # 1. LLM에 가상 답변 생성 요청
    hypo_doc = _generate_hypothetical_doc_llm(query)

    doc_contents = [d["content"] for d in documents]

    # 2. 질문 벡터와 가상 답변 벡터를 각각 생성
    query_vec = embeddings.embed_query(query)
    hyde_vec = embeddings.embed_query(hypo_doc)
    doc_vecs = embeddings.embed_documents(doc_contents)

    # 3. 직접 검색: 질문 ↔ 문서 유사도
    direct_scores = [
        (_cosine_similarity(query_vec, dv), doc)
        for dv, doc in zip(doc_vecs, documents)
    ]

    # 4. HyDE 검색: 가상 답변 ↔ 문서 유사도
    hyde_scores = [
        (_cosine_similarity(hyde_vec, dv), doc)
        for dv, doc in zip(doc_vecs, documents)
    ]

    direct_scores.sort(key=lambda x: x[0], reverse=True)
    hyde_scores.sort(key=lambda x: x[0], reverse=True)

    return {
        "query": query,
        "hypothetical_doc": hypo_doc,
        "direct_results": _to_results(direct_scores),
        "hyde_results": _to_results(hyde_scores),
    }
```

질문 벡터로 검색한 `direct_results` 와 가상 답변 벡터로 검색한 `hyde_results` 가 나란히 반환됩니다. `embeddings` 는 `experiments.py` 가 `jhgan/ko-sroberta-multitask` 모델을 로드해서 넘겨주고, `documents` 는 `data.py` 의 `SEARCH_DOCUMENTS` (사내 규정 10건)입니다. 코드에 등장하는 보조 함수는 같은 파일 위쪽에 미리 정의되어 있습니다.

함수	역할
<code>_generate_hypothetical_doc_llm(query)</code>	LLM에 가상 답변 문서 생성을 요청합니다
<code>_cosine_similarity(a, b)</code>	두 벡터의 코사인 유사도를 계산합니다
<code>_to_results(scored)</code>	(점수, 문서) 리스트를 상위 3개로 잘라 반환합니다

터미널 실험 1-1 실행. HyDE 검색 vs 직접 검색

```
python -m tuning.step1_query_rewrite --step 1-1
```

원리: 가상 답변 문서 생성 → 그 문서와 유사한 실제 문서 검색

쿼리: 연차 신청 절차는 어떻게 됩니까?

가상 문서: 연차 신청에 관한 사내 규정에 따르면, 직원은 사용 예정일 3일 전까지 팀장 승인을 받아야 하며, 1년 근속 시 15일의 유급휴가가 부여됩니다...

직접 검색 결과

순위	내용	유사도
1	연차 신청은 3일 전 서면 제출...	0.5917
2	연차유급휴가는 1년 근속 시 15일...	0.5276
3	재택근무 입사 6개월 이상 신청...	0.4455

3위가 재택근무 문서로, 연차와 무관합니다.

HyDE 검색 결과

순위	내용	유사도
1	연차 신청은 3일 전 서면 제출...	0.6691
2	연차유급휴가는 1년 근속 시 15일...	0.6237
3	재택근무 입사 6개월 이상 신청...	0.5888

모든 점수가 올랐습니다. 1위: 0.5917 → 0.6691

그림 9-6. 실험 1-1. HyDE (Hypothetical Document Embeddings)

직접 검색에서는 1위(0.5917)와 3위(0.4455) 사이 점수 차이가 0.15에 불과해 순위 구분이 어렵습니다. HyDE에서는 가상 답변이 "연차유급휴가"라는 단어를 직접 포함하기 때문에 관련 문서의 유사도가 전반적으로 올랐습니다. 1위 점수가 0.5917 → 0.6691로 상승했고, 상위 문서 간 격차도 뚜렷해졌습니다.

질문을 바꿔서도 실험해 보세요.

터미널 커스텀 쿼리로 HyDE 실험

```
python -m tuning.step1_query_rewrite --step 1-1 --query "쉬는 날 규정"
```

HyDE 실험에서 생각해 볼 것들

- **유사도 점수 차이:** 직접 검색(Direct)과 HyDE 검색의 상위 3개 점수를 비교해 보세요. HyDE 쪽이 전반적으로 높다면, 가상 답변이 질문과 문서 사이의 어휘 차이를 잘 메워 준 겁니다.
- **가상 답변의 품질:** hypothetical_doc 항목을 읽어 보세요. LLM이 만든 가상 답변이 실제 규정과 비슷할수록 검색 정확도가 올라갑니다. 엉뚱한 답변이 나오면 프롬프트를 다듬어야 합니다.
- **순위 변화:** HyDE 검색에서 1위로 올라온 문서가 직접 검색의 1위와 다른지 확인해 보세요. 다르다면 어휘 차이 때문에 직접 검색에서 놓쳤던 문서를 HyDE가 잡아낸 겁니다.

됐다. 오픈이가 터미널을 내려다봤습니다. 유사도가 0.59에서 0.67로 올랐습니다. HyDE는 "쉬는 날"처럼 어휘 차이가 큰 질문에 효과적입니다.

그런데 "복리후생"처럼 의미가 넓은 질문은 어떨까요? 연차도, 재택도, 출장비도 다 걸립니다. 가상 답변 하나로는 한 방향밖에 못 봅니다. 다른 도구를 꺼내 봅니다.

9.4.2 질문을 여러 갈래로 — Multi-Query

동료 : "하나로만 물으면 한 방향밖에 못 보잖아요?"

사서가 서가를 한 방향에서만 훑지 않고 세 갈래로 나눠 찾는 겁니다. LLM에게 "이 질문을 다른 표현으로 3가지 만들어 줘"라고 시키고, 원본 포함 4개 질문으로 각각 검색한 뒤 합집합을 만듭니다.



↓ 합집합 (중복 제거, 최고 점수 유지)

최종 결과: A, B, C, D, E

핵심: 질문 하나를 여러 표현으로 바꾸면 **각 표현이 잡아오는 문서가 다릅니다**. 합집합으로 검색 범위를 넓힙니다.

그림 9-7. Multi-Query 검색 흐름

이 흐름을 코드로 구현합니다. LLM에 프롬프트를 보내 재작성 질문을 받고, 원본 질문과 합쳐 반환하는 함수입니다.

`tuning/step1_query_rewrite/multi_query.py`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 `tuning/step1_query_rewrite/multi_query.py`. 다양한 관점의 쿼리 생성

```
def generate_multi_queries(query, num_queries=3):
    prompt = (
        f"다음 질문을 {num_queries}가지 다른 방식으로 재표현하십시오.\n"
        "각 질문은 같은 정보를 구하지만 다른 표현 방식을 사용해야 합니다.\n"
        "번호 없이 각 질문을 한 줄씩 출력하십시오.\n\n"
        f"원본 질문: {query}\n\n"
        "재표현된 질문들:"
    )
    # TODO: LLM을 호출하고 결과를 줄 단위로 파싱합니다.
    llm_result = _call_llm(prompt)

    # 2. 줄 단위로 파싱, 원본 포함 리스트 구성
    lines = [
        line.strip()
        for line in llm_result.split("\n")
        if line.strip() and not line.strip().startswith("#")
    ]
    queries = [query] + lines[:num_queries]
    return queries
```

원본 질문 1개에 재작성 3개를 합쳐 총 4개 질문이 반환됩니다. `num_queries` (기본값 3)가 생성할 재작성 수이고, `_call_llm(prompt)`은 같은 파일에 정의된 보조 함수로 환경변수 `LLM_PROVIDER`에 따라 Ollama 또는 OpenAI를 호출합니다.

터미널 실험 1-2 실행. Multi-Query 재작성

```
python -m tuning.step1_query_rewrite --step 1-2
```

원리: 1개 질문 → N개 다양한 표현으로 검색 결과 다양화

원본 쿼리: 재택근무 조건과 신청 방법

생성된 쿼리

번호 쿼리	유형
1 재택근무 조건과 신청 방법	원본
2 원격 근무에 관련된 규정 및 절차는?	변형 1
3 원격 근무 신청 방법과 필요한 정보는?	변형 2
4 집으로 출근 조건과 신청 방법은?	변형 3

병합 검색 결과

순위 내용	최고 유사도
1 재택근무는 입사 6개월 이상 정규직에 한해 신청 가능...	0.7355
2 재택근무 중에는 정규 근무시간 준수...	0.6053
3 연차 신청은 3일 전 서면 제출...	0.4531

"재택근무"를 "원격 근무", "집으로 출근" 등으로 바꿔 검색 범위를 넓혔습니다.

그림 9-8. 실험 1-2. Multi-Query

"재택근무 조건"이라는 질문 하나를 "원격 근무 규정", "집으로 출근 조건" 같은 다른 표현으로 바꿔 각각 검색합니다. 4개 질문이 잡아온 문서를 합치면 원본만으로 놓쳤을 문서까지 포함됩니다. 1위 문서(0.7355)는 "재택근무"라는 단어가 직접 들어 있어 원본 질문과 강하게 매칭됐고, 변형 질문들이 같은 문서를 다른 방향에서 보강해 점수가 더 올랐습니다.

재작성 개수를 바꿔서 확장 효과를 비교해 보세요.

터미널 쿼리 수를 5개로 늘려 실험

```
python -m tuning.step1_query_rewrite --step 1-2 --num_queries 5
```

Multi-Query 실험에서 생각해 볼 것들

- **재작성 품질:** LLM이 만든 재작성 질문들을 읽어 보세요. 원본과 같은 의도지만 다른 표현을 쓰고 있어야 합니다. 너무 비슷하면 효과가 떨어집니다.
- **검색 범위 확장:** 합집합 결과에 원본 질문만으로는 못 찾았을 문서가 포함됐는지 확인해 보세요. 새로 들어온 문서가 있다면 Multi-Query의 효과입니다.
- **쿼리 수 조절:** `--num_queries 5` 로 늘리면 검색 범위가 넓어지지만, LLM 호출 횟수도 늘어납니다. 3개면 충분한 경우가 많습니다.

HyDE vs Multi-Query 선택 기준. HyDE는 질문과 문서의 **어휘 차이**가 클 때 효과적입니다. "쉬는 날" → "연차유급휴가"처럼 질문에 쓰는 단어와 문서에 적힌 단어가 다를 때 가상 답변이 다리 역할을 합니다. Multi-Query는 질문의 **의미가 넓을 때** 효과적입니다. "복리후생"처럼 여러 규정에 걸치는 질문을 갈래로 쪼개야 할 때 씁니다. 둘 다 LLM을 호출하므로 응답 시간이 늘어나는 점은 같습니다.

Multi-Query는 넓은 질문에 효과적입니다. 네 갈래로 검색하니 한 방향에서 놓쳤던 문서가 합집합에 올라왔습니다. 오픈이가 팀장에게 화면을 보여주려고 고개를 돌렸습니다.

팀장 : "그거 좋은데요. 근데 제가 아까 'WFH 신청'이라고 물어봤거든요? 아무것도 안 나오더라고요."

이건 다른 종류의 문제입니다. 질문의 넓이가 아니라 **단어 자체**를 모르는 겁니다. 사서는 "재택근무"라는 단어만 알고 있었습니다.

9.4.3 약어를 번역한다 — 약어·동의어 확장

사내에서 흔히 쓰는 약어와 줄임말을 사전으로 만들어 두고, 질문에 포함되면 원래 뜻으로 치환합니다.

`data.py` 에 두 종류의 사전이 들어 있습니다.

참고 tuning/step1_query_rewrite/data.py. 약어·동의어 사전

```
ABBREVIATION_MAP = {
    "WFH": "재택근무(Work From Home)",
    "OT": "초과근무(Overtime)",
    "PIP": "성과개선계획(Performance Improvement Plan)",
    "HR": "인사부서(Human Resources)",
    "4대보험": "국민연금, 건강보험, 고용보험, 산재보험",
    "반차": "반일 연차",
    # ... 외 6개
}

SYNONYM_MAP = {
    "연차": "연차유급휴가",
    "휴가": "연차유급휴가",
    "재택": "재택근무",
    "원격근무": "재택근무",
    "퇴직금": "퇴직급여",
    # ... 외 5개
}
```

`ABBREVIATION_MAP` 은 영문 약어를 한글 풀네임으로, `SYNONYM_MAP` 은 구어 표현을 규정집 정식 명칭으로 바꿉니다. 질문에 "WFH"가 들어오면 "재택근무(Work From Home)"로, "연차"가 들어오면 "연차유급휴가"로 치환됩니다.

이 사전을 순회하며 문자열을 치환하는 함수를 작성합니다.

`tuning/step1_query_rewrite/abbreviation.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 1 tuning/step1_query_rewrite/abbreviation.py. 약어·동의어 확장

```
def expand_abbreviations(query):
    # TODO: 쿼리의 약어와 동의어를 확장합니다.
    expanded = query
    applied = []

    # 1. 약어 확장: "WFH" → "재택근무(Work From Home)"
    for abbrev, full_form in ABBREVIATION_MAP.items():
        if abbrev in expanded:
            expanded = expanded.replace(abbrev, full_form)
            applied.append(f"약어 '{abbrev}' -> '{full_form}'")

    # 2. 동의어 확장: "휴가" → "연차유급휴가"
    for term, synonym in SYNONYM_MAP.items():
        if term in expanded and synonym not in expanded:
            expanded = expanded.replace(term, synonym)
            applied.append(f"동의어 '{term}' -> '{synonym}'")

    return {"original": query, "expanded": expanded, "applied": applied}
```

ABBREVIATION_MAP 과 SYNONYM_MAP 을 순회하며 문자열 치환을 수행합니다. applied 리스트에 어떤 변환이 적용됐는지 기록하므로, "WFH 신청 방법"이 "재택근무(Work From Home) 신청 방법"으로 바뀌는 과정을 터미널에서 확인할 수 있습니다.

터미널 실험 1-3 실행. 약어·동의어 확장

```
python -m tuning.step1_query_rewrite --step 1-3
```

원리: 사전 기반 매핑으로 약어와 동의어를 정규 표현으로 치환

원본 쿼리	확장된 쿼리	적용 규칙
WFH 정책이 어떻게 됩니까?	재택근무(Work From Home) 정책이...	약어 WFH → 재택근무
OT 신청 절차를 알려주십시오	초과근무(Overtime) 신청 절차를...	약어 OT → 초과근무
연차 신청 방법	연차유급휴가 신청 방법	동의어 연차 → 연차유급휴가
4대보험 가입 시기	국민연금, 건강보험, 고용보험, 산재보험...	약어 4대보험 → 풀네임

사전에 등록된 약어와 동의어가 자동으로 치환됩니다.

그림 9-9. 실험 1-3. 약어/동의어 확장

"WFH"라고 물었는데 문서에는 "재택근무"라고 적혀 있으면 벡터 검색에서도 놓칠 수 있습니다. 사전이 이 간극을 메워 줍니다. LLM 호출 없이 단순 문자열 치환이라 응답 지연이 없습니다.

약어가 포함된 질문을 직접 넣어 보세요.

터미널 커스텀 쿼리로 약어 확장 실험

```
python -m tuning.step1_query_rewrite --step 1-3 --query "OT 신청 절차"
```

약어 확장 실험에서 생각해 볼 것들

- **적용된 변환:** applied 목록에 어떤 규칙이 적용됐는지 확인해 보세요. 약어(WFH → 재택근무)와 동의어 (휴가 → 연차유급휴가) 두 종류가 있습니다.
- **검색 결과 변화:** 확장 전후로 검색 결과가 달라지는지 비교해 보세요. 문서에 "재택근무"라고 적혀 있는데 질문에 "WFH"를 쓰면 벡터 검색에서도 놓칠 수 있습니다.
- **사전 관리:** data.py 에 없는 약어를 넣으면 그대로 통과합니다. 사내에서 자주 쓰는 약어를 사전에 추가하면 바로 효과가 나옵니다. 슬랙에서 검색 실패 로그를 모니터링하면 "이 단어가 빠져 있구나"를 알 수 있습니다. 사전이 커질수록 검색 실패율이 떨어집니다.

질의 변환 도구 세 가지를 실험했습니다. 질문을 바꾸는 쪽은 여기까지입니다. 그런데 오픈이가 검색 결과 자체를 살펴봤습니다. 가져온 청크 3개를 나란히 놓으니 "연차 신청은 3일 전 서면으로", "팀장 승인이 필요합니다", "긴급한 경우 구두 신청 가능". 각각은 맞는 말인데, 조각조각 끊겨 있습니다.

질문을 잘 바꿔서 찾았는데, 찾아온 결과를 다듬는 도구도 필요하겠는데.

9.5 쪽지가 아니라 규정집을 펼친다 — 결과 가공 (step2)

동료 : "조각 3개 가져왔는데, 각각 한두 줄이면 LLM이 전체 맥락을 알 수 있어요?"

청크 여러 개를 가져와도 각 청크가 짧은 조각이면 LLM은 규정의 전체 흐름을 볼 수 없습니다. 질의 변환이 질문을 다듬는 작업이었다면, 결과 가공은 LLM에 넘기는 문서를 다듬는 작업입니다.

실습 흐름

1. 자식 청크로 검색하고 부모 문서 전체를 반환합니다 (`--step 2-1`)
2. 검색 후 관련 문장만 압축해서 반환합니다 (`--step 2-2`)
3. 질문에서 메타데이터 조건을 추출해 검색 대상을 좁힙니다 (`--step 2-3`)

9.5.1 원본 문서 꺼내기 — Parent Document Retriever

작은 청크로 검색하되, LLM에게는 그 청크가 속한 **부모 문서 전체**를 넘깁니다. 정확도(작은 청크)와 맥락(큰 문서)을 동시에 취하는 방식입니다.

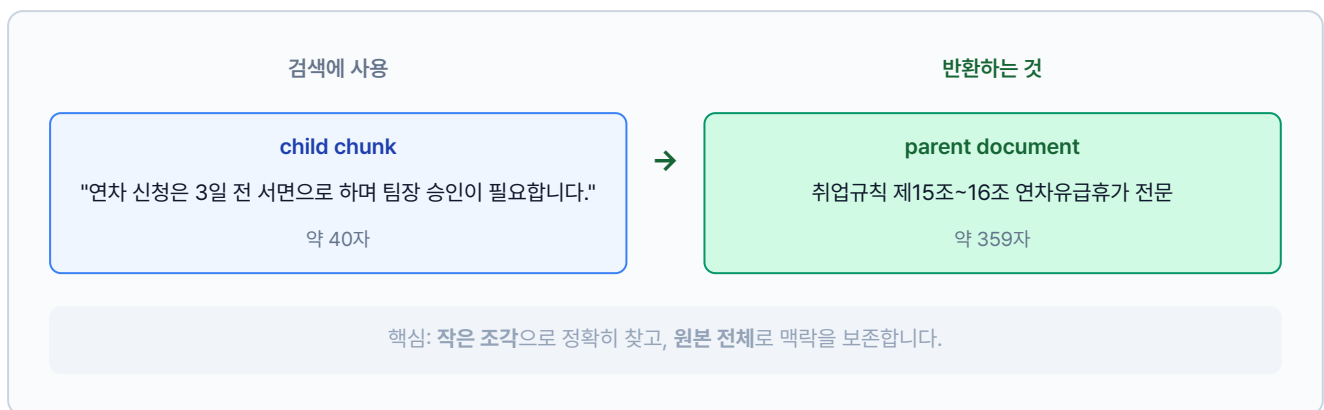


그림 9-10. Parent Document Retriever 동작 원리

이 흐름을 코드로 구현합니다. 자식 청크별 유사도를 계산하고, 매칭된 조각이 속한 부모 문서 전체를 반환하는 클래스입니다.

`tuning/step2_advanced_retriever/parent_doc.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2

tuning/step2_advanced_retriever/parent_doc.py. Parent Document Retriever

```

class ParentDocumentRetriever:
    def __init__(self, parent_docs, child_chunks, embeddings=None):
        self.parent_docs = {doc["id"]: doc for doc in parent_docs}
        self.child_chunks = child_chunks
        self.embeddings = embeddings

    def search(self, query, top_k=2):
        # TODO: 자식 청크로 검색하고 부모 문서를 반환합니다.

        # 1. 자식 청크별 유사도 계산
        query_vec = self.embeddings.embed_query(query)
        scored_chunks = []
        for vec, chunk in zip(self._child_vectors, self.child_chunks):
            score = _cosine_similarity(query_vec, vec)
            scored_chunks.append((score, chunk))

        scored_chunks.sort(key=lambda x: x[0], reverse=True)

        # 2. 중복 제거: 같은 부모는 한 번만
        seen_parents = set()
        results = []
        for score, chunk in scored_chunks:
            parent_id = chunk["parent_id"]
            if parent_id not in seen_parents and len(results) < top_k:
                seen_parents.add(parent_id)
                parent_doc = self.parent_docs.get(parent_id)
                # 3. 자식 청크 + 부모 문서 전체를 함께 반환
                results.append({
                    "parent_content": parent_doc["content"],
                    "child_chunk": chunk["content"],
                    "score": round(score, 4),
                })
        return results

```

작은 `child_chunk` 로 유사도를 계산하되, 결과에는 `parent_content` 전체가 담깁니다. `parent_docs` 는 `data.py` 의 원본 문서 리스트(ID, 제목, 전체 내용 포함)이고, `child_chunks` 는 원본을 잘게 쪼개 조각 리스트입니다. 각 조각에는 자신이 속한 부모의 ID(`parent_id`)가 들어 있습니다. `self._child_vectors` 는 `__init__` 에서 모든 자식 청크를 미리 임베딩해 둔 벡터입니다. 같은 부모가 중복 반환되지 않도록 이미 꺼낸 부모는 건너뛵니다.

터미널

실험 2-1 실행. Parent Document Retriever

```
python -m tuning.step2_advanced_retriever --step 2-1
```

원리: 작은 청크로 검색 → 원본 전체 문서 반환 (컨텍스트 풍부)

쿼리: 연차 신청 절차

순위	매칭 청크	부모 문서	부모 길이	유사도
1	연차 신청은 3일 전 서면으로...	취업규칙 제15조~16조 연차유급휴가	359자	0.6272
2	재택근무는 입사 6개월 이상...	재택근무 운영지침	269자	0.4314

매칭 청크는 한 문장이지만, 반환된 부모 문서는 359자 전문입니다.

그림 9-11. 실험 2-1. ParentDocumentRetriever

"연차 신청은 3일 전 서면으로"라는 짧은 조각이 매칭됐지만, 실제로 반환된 건 취업규칙 제15조~16조 전문(359자)입니다. LLM이 "3일 전까지 뭘 해야 하는지" 앞뒤 맥락까지 볼 수 있습니다.

검색 결과 수를 바꿔서 맥락의 양을 조절해 보세요.

터미널 커스텀 쿼리 + top_k 변경

```
python -m tuning.step2_advanced_retriever --step 2-1 --query "병가 규정" --top_k 3
```

부모 문서 검색 실험에서 생각해 볼 것들

- **크기 차이:** child_chunk(매칭된 작은 조각)와 parent_content(반환된 원본)의 길이를 비교해 보세요. 조각은 100~200자인데 원본은 500자 이상일 겁니다. 이 차이만큼 LLM이 더 넓은 맥락을 볼 수 있습니다.
- **맥락 복원:** 작은 조각만 읽었을 때 의미가 불완전하지 않은지 확인해 보세요. "3일 전까지"라는 조각만으로는 뭘 3일 전까지 해야 하는지 모르지만, 원본에는 앞뒤 문맥이 있습니다.
- **top_k 조절:** `--top_k 5`로 늘리면 더 많은 부모 문서를 가져오지만, LLM에 넘기는 컨텍스트가 길어집니다. 문서 3개면 대부분 충분합니다.

맥락이 살아났습니다. 359자짜리 규정 전문이 LLM에 들어가니 "3일 전까지 뭘 해야 하는지" 앞뒤가 보입니다. 그런데 오픈이가 토큰 사용량을 확인했습니다. 부모 문서를 통째로 넣으니 프롬프트가 눈에 띄게 길어졌습니다.

동료: "규정 전문이 다 필요해요? 질문이랑 관련 없는 조항도 섞여 있잖아요."

9.5.2 핵심 문장만 남기기 — Contextual Compression

부모 문서를 통째로 가져오면 맥락은 살지만 쓸데없는 문장이 섞입니다. 사서가 규정집을 펼쳐 보여주되, 형광펜으로 질문과 관련된 문장만 밑줄 긋고 나머지는 접어 버리는 겁니다.

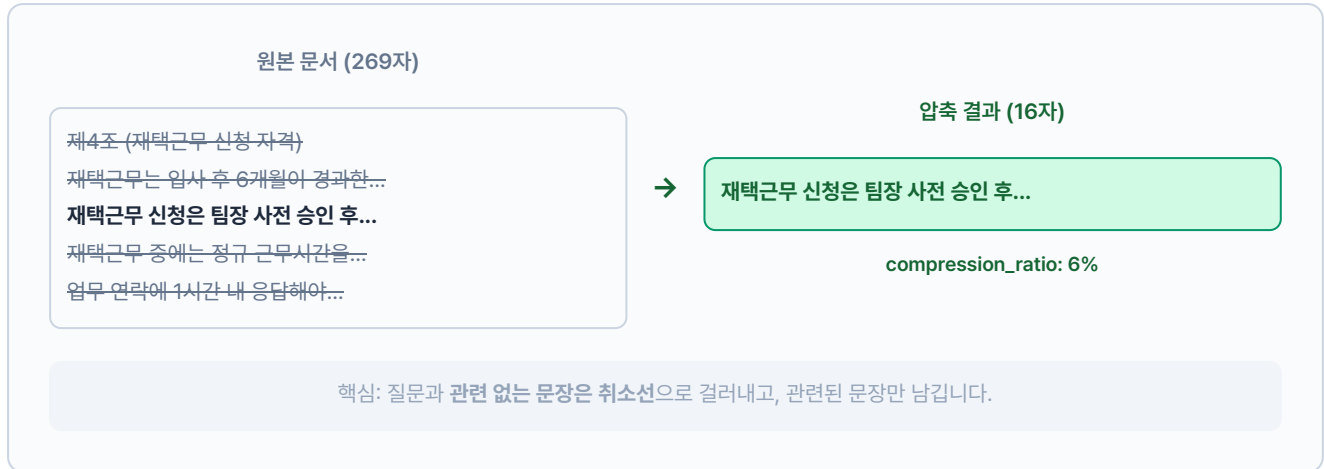


그림 9-12. Contextual Compression 동작 원리

이 흐름을 코드로 구현합니다. 문서를 문장 단위로 쪼개고, 질문 단어와 겹치는 문장만 골라내는 클래스입니다.

`tuning/step2_advanced_retriever/compression.py` 를 열고 TODO의 `pass` 를 지우고 아래 코드를 작성합니다.

실습 2 tuning/step2_advanced_retriever/compression.py. Contextual Compression

```

class ContextualCompressionRetriever:
    def __init__(self, documents, embeddings=None):
        self.documents = documents
        self.embeddings = embeddings

    def search(self, query, top_k=3):
        # TODO: 검색 결과에서 질문과 관련된 문장만 압축합니다.
        # 1. 유사도 기준으로 문서 정렬
        query_vec = self.embeddings.embed_query(query)
        doc_vecs = self.embeddings.embed_documents(
            [d["content"] for d in self.documents]
        )
        scored = [
            (_cosine_similarity(query_vec, dv), doc)
            for dv, doc in zip(doc_vecs, self.documents)
        ]
        scored.sort(key=lambda x: x[0], reverse=True)

        results = []
        for score, doc in scored[:top_k]:
            original = doc["content"]
            # 2. 핵심 문장만 추출 (최대 3문장)
            compressed = _compress(query, original)
            results.append({
                "original_content": original,
                "compressed_content": compressed,
                "compression_ratio": round(len(compressed) / len(original), 2),
            })
        return results

```

`_compress()` 함수가 문서를 문장 단위로 쪼갠 뒤 질문 단어와 겹치는 문장만 골라냅니다.

`compression_ratio` 가 0.3이면 원본의 70%를 덜어낸 것이니, 부모 문서를 통째로 가져와도 프롬프트 토큰을 크게 절약할 수 있습니다.

터미널 실험 2-2 실행. Contextual Compression

```
python -m tuning.step2_advanced_retriever --step 2-2
```

원리: 검색된 문서에서 쿼리 관련 부분만 추출 → LLM 토큰 절약

쿼리: 연차 신청 절차

순위	원본 길이	압축 길이	압축률	압축된 내용
1	269자	16자	6%	제4조 (재택근무 신청 자격)...
2	359자	34자	9%	긴급한 경우에는 구두 신청 후 사후 서면 제출...

269자 문서가 16자로 줄었습니다. 질문과 관련된 문장만 남깁니다.

그림 9-13. 실험 2-2. ContextualCompressionRetriever

359자짜리 연차 규정 문서가 34자로 줄었습니다. "연차 신청 절차"와 관련된 문장만 남기고 나머지는 잘라낸 겁니다. LLM에 넘기는 컨텍스트가 짧아지니 토큰 비용이 줄고 답변 정확도도 올라갑니다.

압축 검색 실험에서 생각해 볼 것들

- **압축률:** compression_ratio 수치를 확인해 보세요. 0.3이면 원본의 30%만 남긴 겁니다. 질문과 무관한 부분을 70% 덜어낸 셈입니다.
- **정보 보존:** compressed_content를 읽어 보세요. 질문에 답하는 데 필요한 핵심 정보가 남아 있어야 합니다. 너무 짧으면 중요한 내용이 잘려 나갔을 수 있습니다.
- **LLM 부담 감소:** 원본 500자를 150자로 줄이면 LLM이 읽을 양이 줄어듭니다. 문서 10개를 넘기는 상황에서 효과가 두드러집니다.

Parent Doc + Compression 조합. 이 둘은 세트로 쓸 때 효과가 큼니다. Parent Doc이 넓은 맥락을 가져오고 Compression이 핵심만 남기는 구조입니다. Parent Doc 단독 사용 시 부모 문서가 길어 토큰 비용이 늘어나는 문제를 Compression이 해결합니다.

Parent Doc과 Compression은 검색 결과의 양과 질을 조절하는 도구입니다. 마지막으로 하나 더. 오픈이가 동료의 질문 하나를 떠올렸습니다. "인사팀 관련 규정만 보고 싶은데." 이 질문에는 **부서명**이라는 조건이 섞여 있습니다. 벡터 유사도만으로는 "인사팀"이라는 조건을 걸 수 없습니다. 이런 유형의 질문이 들어온다면 쓸 수 있는 도구가 있습니다.

9.5.3 조건이 섞인 질문 — Self-Query Retriever

"2025년 인사팀 문서만" 같은 조건이 섞인 질문에서 **메타데이터 필터**를 자동으로 추출할 수 있습니다. 질문에 포함된 키워드를 감지해 `topic=인사팀`, `year=2025` 같은 조건을 만들고, 전체 문서 중 조건에 맞는 문서만 남긴 뒤 유사도 검색을 수행합니다.

"인사팀 관련 규정"이라는 질문이 들어오면 `{"topic": "인사"}` 필터가 자동으로 만들어지고, 수백 건의 문서 중 인사 관련 문서만 추려서 검색 대상으로 삼습니다. `topic_keywords` 는 `data.py` 에 정의된 토픽-키워드 매핑 딕셔너리로, `{"인사": ["인사", "인사팀", "hr"], "보안": ["보안", "정보보호"]}` 같은 구조입니다.

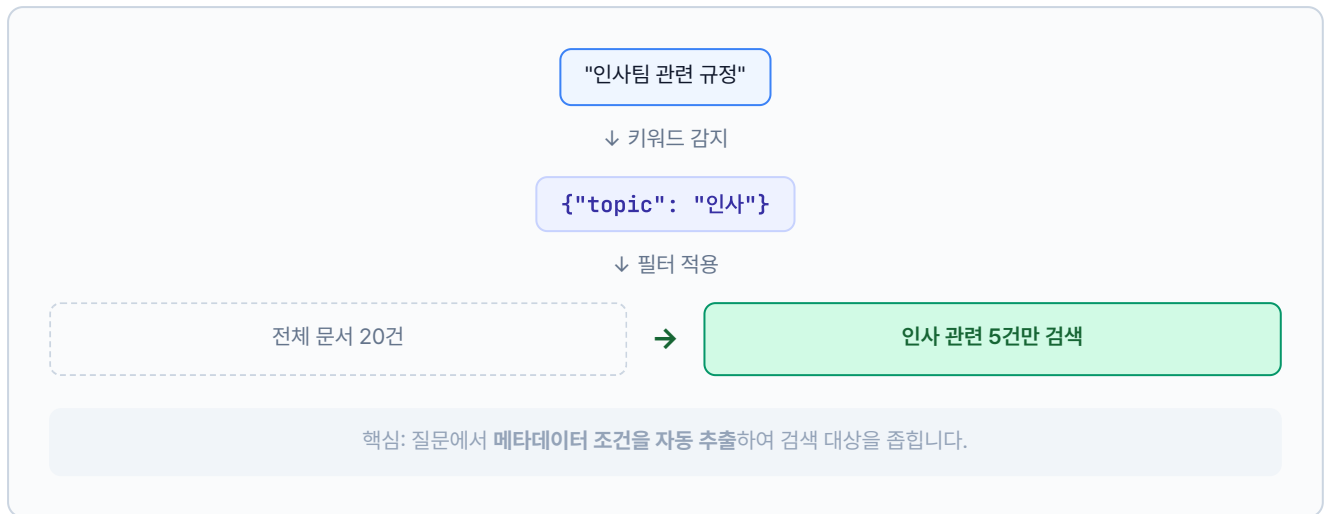


그림 9-14. Self-Query Retriever 동작 원리

설명 tuning/step2_advanced_retriever/self_query.py. Self-Query 필터 추출 + 검색

```
# self_query.py - 쿼리에서 메타데이터 조건을 자동 추출합니다
class SelfQueryRetriever:
    def __init__(self, documents, topic_keywords=None, embeddings=None):
        self.documents = documents
        self.topic_keywords = topic_keywords or {}

    def extract_filter(self, query):
        # 1. 질문에서 키워드를 감지해 메타데이터 필터 생성
        filters = {}
        query_lower = query.lower()
        for topic, keywords in self.topic_keywords.items():
            if any(kw in query_lower for kw in keywords):
                filters["topic"] = topic
                break
        return filters

    def search(self, query, top_k=3):
        filters = self.extract_filter(query)
        # 2. 필터에 맞는 문서만 남김
        filtered = self.documents
        for key, value in filters.items():
            filtered = [
                doc for doc in filtered
                if doc.get("metadata", {}).get(key) == value
            ]
        # 3. 남은 문서에서 유사도 검색
        scored = []
        for doc in filtered:
            # ... 유사도 계산 (생략)
            scored.append((score, doc))
        return [{"content": doc["content"], "applied_filter": filters}
                for _, doc in scored[:top_k]]
```

`extract_filter()` 가 질문에서 부서명이나 연도 같은 조건을 뽑아내고, `search()` 가 그 조건에 맞는 문서만 추려서 유사도 검색을 수행합니다. 전체 코드는 완성본 레포(ai-qa-lag)를 참고하세요.

터미널 실험 2-3 실행. Self-Query Retriever

```
python -m tuning.step2_advanced_retriever --step 2-3
```

원리: 질문에서 메타데이터 필터 자동 추출 → 정확한 필터링

쿼리: 재택근무 조건

자동 추출된 필터: **topic: 재택근무**

순위	내용	토픽	유사도
1	제4조 (재택근무 신청 자격) 입사 후 6개월이 경과한 정규직...	재택근무	0.7136

topic: 재택근무 필터가 자동 추출되어 해당 문서만 검색됩니다.

그림 9-15. 실험 2-3. SelfQueryRetriever

"재택근무 조건"이라는 질문에서 **topic: 재택근무** 필터가 자동으로 추출됐습니다. 전체 문서를 벡터 검색하는 대신 재택근무 관련 문서만 대상으로 좁혀 검색하니 무관한 문서가 섞이지 않습니다.

부서명이 포함된 질문을 넣어서 필터가 정확히 잡히는지 확인해 보세요.

터미널 커스텀 쿼리로 필터 추출 실험

```
python -m tuning.step2_advanced_retriever --step 2-3 --query "인사팀 관련 규정"
```

Self-Query 실험에서 생각해 볼 것들

- **필터 추출:** applied_filter에 어떤 메타데이터 조건이 추출됐는지 확인해 보세요. "인사팀 관련 규정"이라고 물으면 **topic: 인사** 처럼 부서명이 필터로 잡혀야 합니다.
- **검색 범위 축소:** 필터 적용 전후로 검색 대상 문서 수가 줄어드는지 비교해 보세요. 전체 20건에서 5건으로 줄어들면 그만큼 정확도가 올라갑니다.
- **필터가 안 잡히는 경우:** 질문에 메타데이터 단서가 없으면 필터 없이 일반 검색으로 넘어갑니다. "연차 신청 방법"처럼 부서나 카테고리 힌트가 없는 질문이 이 경우입니다.

다섯 가지 도구를 실험했습니다. 오픈이가 동료들의 질문 패턴을 다시 펼쳐 봤습니다. 약어를 많이 쓰고, 근거로 규정 전문을 보고 싶어합니다. 약어 사전과 Parent Doc — 이 두 개면 충분합니다.

전부 켤 필요 없어. 아픈 데만 약을 바르면 돼.

9.6 사서가 달라졌다

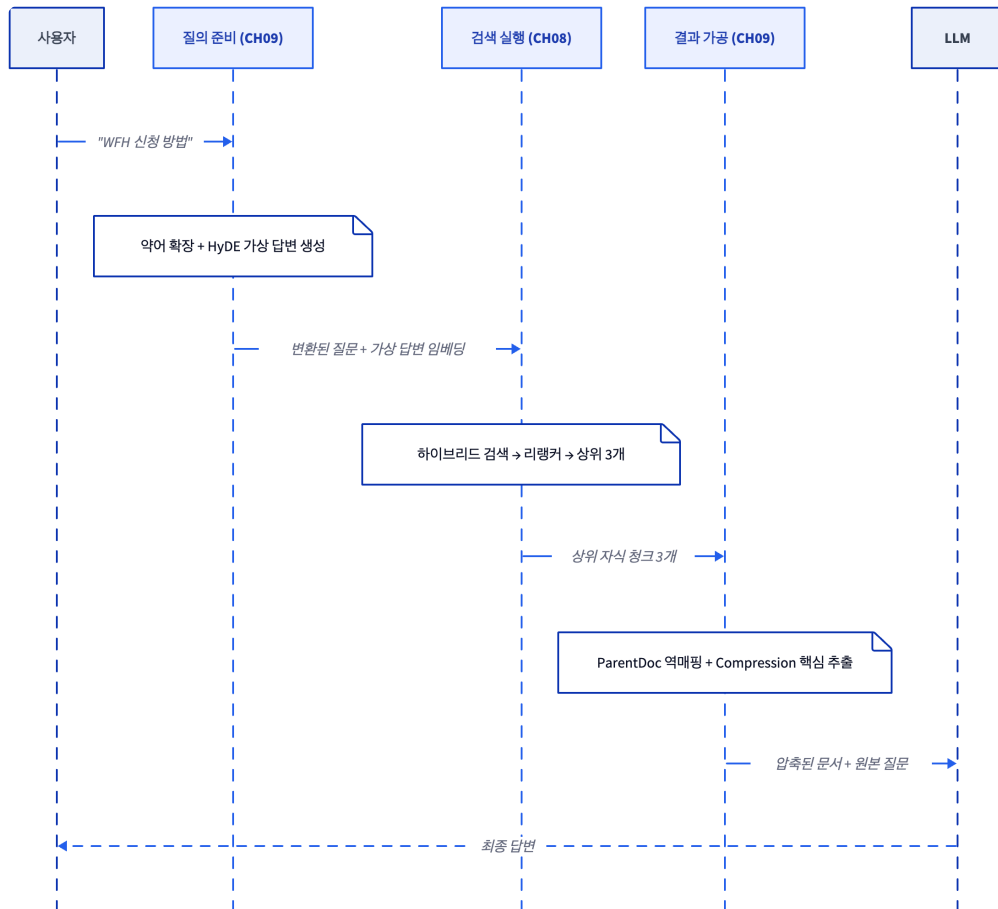


그림 9-16. 챕터 9 파이프라인. 질문을 변환(확장, 재작성, HyDE)한 뒤 검색하고, 결과를 가공(Parent, Compression)해 LLM에 넘깁니다

동료에게 다시 써 보라고 했습니다. 동료가 키보드를 두드렸습니다.

동료 : "WFH 정책이 뭐야?"

"재택근무" 문서를 정확히 가져와 답변을 내놓습니다. 약어 사전이 "WFH"를 "재택근무(Work From Home)"로 바꿔 놓은 덕이었습니다.

동료 : "쉬는 날 규정 알려줘."

취업규칙 제15조 연차유급휴가 원본 문서 전체가 근거로 올라옵니다. 동의어 확장이 "휴가"를 "연차유급휴가"로 연결하고, Parent Doc이 원본 문서를 통째로 꺼내 왔습니다.

동료 : "...이번엔 진짜 되네요."

오픈이가 의자를 돌려 동료를 봤습니다.

오픈이 : "약어 사전이랑 Parent Doc만 켜거든요. 우리 동료들이 약어를 많이 쓰고, 근거로 규정 전문을 보고 싶어하니까요. 나머지는 필요할 때 켜면 돼요."

다섯 개 중 두 개. 검색 엔진은 안 건드렸는데 이렇게 달라진다.

의자가 삐걱거렸습니다. 팀장이 자리에서 일어나며 화이트보드를 훑듯 봤습니다.

팀장 : "텍스트는 됐는데요. PDF 속 표랑 이미지는?"

오픈이 : "표요?"

팀장 : "규정집에 표가 있잖아요. 그건 텍스트 검색으로 잡히나요?"

9.7 튜닝 구성도

챕터 8에서는 검색 엔진 안쪽을 손봤습니다. 청크를 자르는 방법, 결과를 다시 정렬하는 방법, 두 검색기를 섞는 방법까지 세 종류였습니다. 이번 챕터에서 다룬 다섯 가지는 그 검색 엔진의 **앞뒤**를 다듬는 실험입니다. 질문이 검색기에 닿기 전에 모양을 바꾸는 쪽(HyDE, Multi-Query, 약어·동의어 확장)과, 검색 결과를 LLM에 넘기기 전에 다듬는 쪽(Parent Document, Contextual Compression)으로 나뉩니다.

두 챕터를 합치면 누적 여덟 개의 튜닝이 메뉴판에 채워집니다. 챕터 10에서는 텍스트로는 잡히지 않는 영역(이미지, 표)과 품질을 숫자로 재는 평가 프레임워크가 더해질 예정입니다.

모든 튜닝을 한꺼번에 적용할 필요는 없습니다. 문서 성격과 질문 패턴에 맞춰 필요한 것부터 하나씩 골라 씁니다.

CH 09 · 질문을 제대로 이해 못한다 - 튜닝 메뉴판 (CH08·09 누적 8종)

CHAPTER 09 · TUNING MENU

RAG 튜닝 메뉴판

CH08 검색 3종 + CH09 질의 5종 = 누적 8종 · CH10에서 3종 추가 예정

	튜닝	무엇을 바꾸나	언제 쓰나	챕터
✓	칭킹 전략	문서를 자르는 방법 (Fixed → Recursive → Semantic)	주제가 섞이는 문제가 있을 때	8
✓	리랭킹	가져온 문서를 Cross-Encoder로 재정렬	검색 결과 상위에 엉뚱한 문서가 섞일 때	8
✓	하이브리드 검색	벡터 + BM25를 alpha 가중합으로 합침	의미 검색과 키워드 검색 둘 다 필요할 때	8
✓	HyDE	가상 답변을 먼저 생성해 답변-문서 유사도로 검색	질문과 문서의 어휘 차이가 클 때	9
✓	Multi-Query	한 질문을 여러 표현으로 재작성해 합집합 검색	질문의 의미가 넓어 한 방향만으로 부족할 때	9
✓	약어·동의어 확장	사내 용어 사전으로 약어를 원 표현으로 치환	사내 약어, 줄임말 검색 실패가 잦을 때	9
✓	Parent Document	작은 청크로 검색하되 부모 문서 단위로 반환	조각만 보면 맥락이 부족할 때	9
✓	Contextual Compression	검색된 문서에서 질문과 무관한 부분 제거	부모 문서가 길어 토큰 비용이 부담될 때	9
—	Vision LLM	이미지·표를 Vision 모델로 읽어 텍스트 변환	PDF 속 표, 차트, 이미지가 검색에 빠질 때	10
—	하이브리드 파서	텍스트 + 시각 요소를 함께 파싱하는 파이프라인	복합 문서(텍스트+표+이미지)를 처리할 때	10
—	RAG 평가 프레임워크	검색 정확도, 답변 충실도를 숫자로 측정	튜닝 전후 품질을 객관적으로 비교할 때	10

✓ 완료 (진한 행 = 이번 챕터) · — 앞으로 다룰 항목

CH09는 질문 쪽 다섯 실험 - HyDE · Multi-Query · 약어 · Parent Doc · Compression 추가. 누적 8종이 CH10에서 파이프라인으로 선별 조립됨

그림 9-18. 챕터 9의 결과. 메뉴판에 질의 5종(HyDE·Multi-Query·약어·Parent Doc·Compression)이 새로 채워져 누적 8종이 되었습니다

용어 정리

본문 속 표현	진짜 용어	정식 정의
"단어를 번역"	약어·동의어 확장	사내 용어 사전을 이용해 약어를 원 표현으로 치환
"답을 먼저 상상"	HyDE	질문에 대한 가상 답변을 LLM이 먼저 생성하고, 그 답변으로 검색
"여러 갈래로 물어보기"	Multi-Query	한 질문을 LLM이 여러 표현으로 재작성해 각각 검색 후 합집합
"원본 문서 전체 꺼내기"	Parent Document Retriever	작은 청크로 검색하되 반환은 큰 부모 문서 단위
"핵심 문장만 추리기"	Contextual Compression	검색된 문서에서 질문과 무관한 부분을 LLM이 제거
"조건을 필터로 변환"	Self-Query Retriever	LLM이 질문에서 메타데이터 조건을 뽑아 <code>where</code> 필터로 심음

이것만은 기억하자

- 검색의 앞뒤를 다듬으면 같은 엔진이 더 똑똑해집니다. 챗터 8이 엔진 자체를 바꿨다면 챗터 9는 입력(쿼리)과 출력(결과)을 가공합니다. 인프라 변경 없이 체감 품질이 가장 크게 오르는 구간입니다.
- 약어, 동의어 사전은 무조건 만드세요. 사내 용어는 공식 문서와 일상 표현이 다릅니다. "WFH ↔ 재택근무" 같은 매핑 한 줄이 실패 질문을 크게 줄여 줍니다.
- Parent Doc + Compression 조합이 안정적입니다. 정확도는 작은 청크, 맥락은 부모 문서, 프롬프트 비용은 압축으로 잡는 전략입니다.
- 챗터 10에서는 텍스트로 못 잡는 이미지와 표를 Vision LLM으로 읽고, RAG 전체 품질을 숫자로 측정합니다.